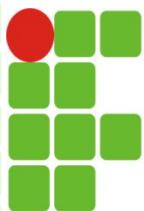


INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SUL-RIO-GRANDENSE
Campus Passo Fundo

ALGORITMOS II

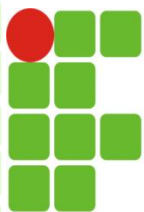
Prof. Adilso Nunes de Souza



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SUL-RIO-GRANDENSE
Campus Passo Fundo

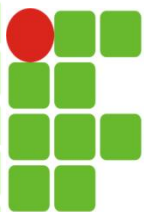
ROTEIRO

- **Aritmética de ponteiro**



TAMANHO EM BYTES

- Ao pensar em aritmética de ponteiros é necessário conhecer o tamanho em bytes de cada tipo de dado:
 - Char: 1 byte
 - Int: 4 bytes
 - Float: 4 bytes
 - Double: 8 bytes
- O operador `sizeof()` pode ser utilizado para identificar o tamanho em bytes que uma variável ocupa ou um tipo de dado.

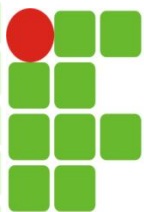


TAMANHO EM BYTES

```
int x = 0;  
cout << sizeof(char) << endl;  
cout << sizeof(x) << endl;  
cout << sizeof(float) << endl;  
cout << sizeof(double) << endl;
```

Saída:

1
4
4
8



ORDEM DE PRECEDÊNCIA

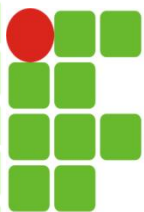
- Outra observação importante na manipulação de ponteiros é a ordem de precedência dos operadores.

Ex.

```
int x = 234;  
int *px;  
px = &x;  
*px++;  
cout << *px << endl;
```

Saída

Lixo de memória



ORDEM DE PRECEDÊNCIA

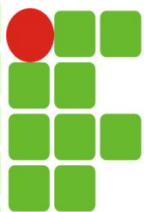
- O operador de incremento “++” tem maior prioridade que o operador “*” por isso a necessidade dos parênteses.

Ex.

```
int x = 234;  
int *px;  
px = &x;  
(*px)++;  
cout << *px << endl;
```

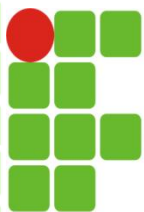
Saída

235



OPERADORES PERMITIDOS

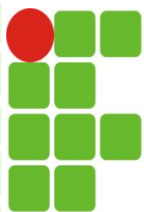
- Só é permitido a utilização de quatro operadores aritméticos com ponteiros:
 - Incremento (++)
 - Decremento (--)
 - Adição (+)
 - Subtração (-)



ARITMÉTICA DE PONTEIRO

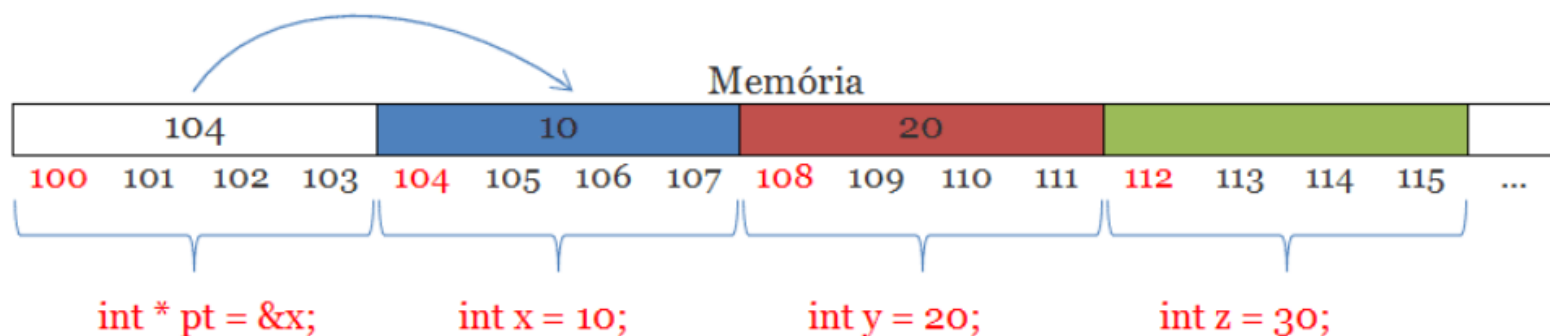
```
int x;  
int *px;  
px = &x;  
px++;
```

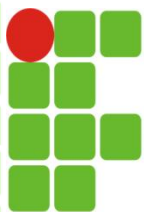
- Supondo que o endereço de memória de X é 100, ao incrementar o ponteiro px ele passa a referenciar o endereço de memória 104, ou seja o próximo endereço inteiro. O mesmo vale para o decremento.



ARITMÉTICA DE PONTEIRO

Conhecendo as operações básicas de incremento e decremento de ponteiros, precisamos entender o que ocorre com o endereço armazenado no ponteiro quando uma das operações aritméticas é utilizada.

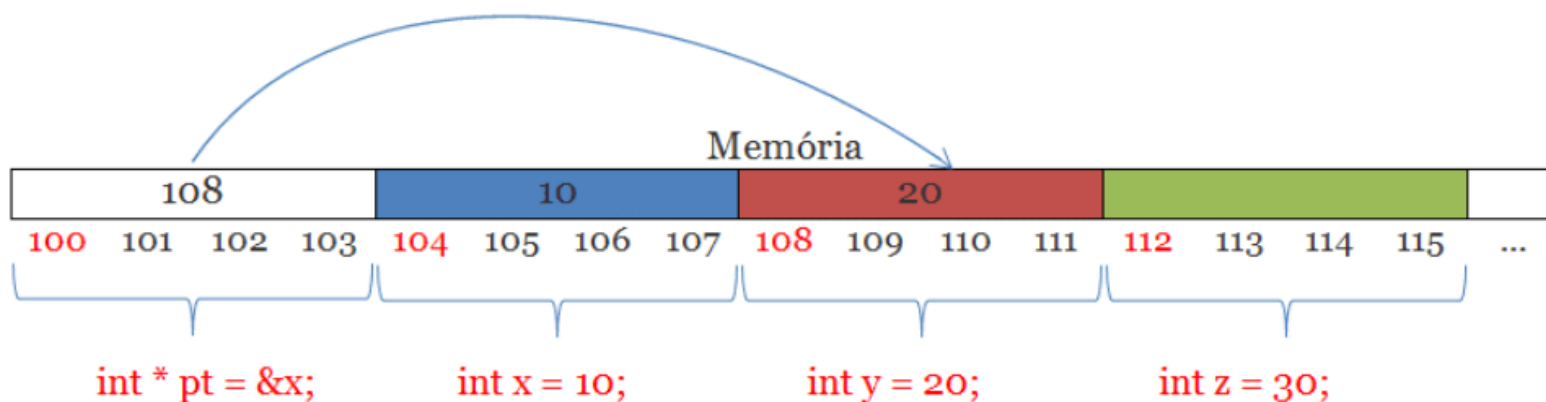


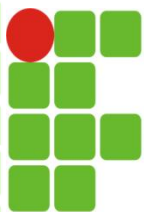


ARITMÉTICA DE PONTEIRO

Executando o incremento do ponteiro `pt`:
`pt++;`

O ponteiro passa a apontar para a localização da memória do próximo elemento do seu tipo base, como pode ser observado:





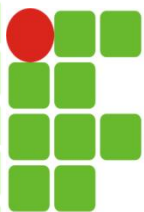
ARITMÉTICA DE PONTEIRO

Também não se limita apenas a operações de incremento e decremento, sendo possível adicionar ou subtrair números inteiros, exemplo:

$$pt = pt + 4;$$

$4 \times \text{sizeof}(\text{int}) = 16 \text{ bytes}$

neste caso o ponteiro desloca-se 16 bytes na memória.



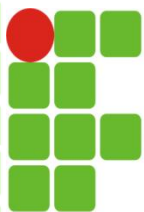
ARITMÉTICA DE PONTEIRO

O que nos permite também calcular a diferença entre dois endereços de memória:

```
int v[10];  
int *p1 = &v[2];  
int *p2 = &v[8];  
cout << p1 << endl;  
cout << p2 << endl;  
cout << p2 - p1 << endl;
```

Saída

```
0x61fe18  
0x61fe30  
6
```



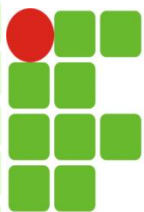
ARITMÉTICA DE PONTEIRO

Já a soma entre dois endereços de memória é uma operação não permitida:

```
int v[10];  
int *p1 = &v[2];  
int *p2 = &v[8];  
cout << p1 << endl;  
cout << p2 << endl;  
cout << p2 + p1 << endl;
```

Saída

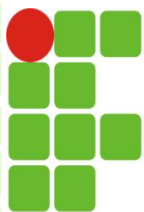
**error: invalid operands of types 'int*' and
'int*' to binary 'operator+' |**



ARITMÉTICA DE PONTEIRO

Nestes exemplos consideramos os endereços de memória especificados, mas na prática os endereços só são consecutivos tratando-se de vetores.

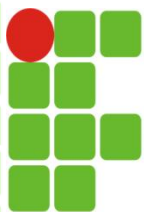
```
int *px = new int [5]{1,2,3,4,5};  
int *pi = new int(0);  
while(*pi < 5)  
{  
    cout << px << endl; //mostra o endereço atual do px  
    cout << *px << endl;  
    px++;  
    (*pi)++;  
}
```



ARITMÉTICA DE PONTEIRO

Após incrementar um ponteiro de um array deve-se observar que, ao final do percurso do array, o ponteiro não está mais no endereço reservado ao vetor.

```
int *px = new int [5]{1,2,3,4,5};  
int *pi = new int(0);  
cout << "Endereco inicial do vetor: " << px << endl;  
while(*pi < 5)  
{  
    cout << px << endl;//mostra o endereço atual do px  
    cout << *px << endl;  
    px++;  
    (*pi)++;  
}  
  
cout << "Endereco final do vetor: " << px << endl;
```



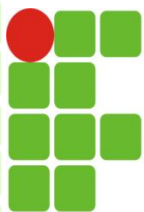
ARITMÉTICA DE PONTEIRO

Para retornar ao endereço inicial do array basta decrementar, a mesma quantidade de bytes que foram incrementadas. Ver exemplo 25 e 26

```
int *px = new int [5]{1,2,3,4,5};
int *pi = new int(0);
cout << "Endereco inicial do vetor: " << px << endl;
while(*pi < 5)
{
    cout << px << endl;//mostra o endereço atual do px
    cout << *px << endl;
    px++;
    (*pi)++;
}

cout << "Endereco final do vetor: " << px << endl;
px -= *pi;
cout << "Volta ao endereco inicial: " << px << endl;
```

```
Endereco inicial do vetor: 0xe81790
0xe81790
1
0xe81794
2
0xe81798
3
0xe8179c
4
0xe817a0
5
Endereco final do vetor: 0xe817a4
Volta ao endereco inicial: 0xe81790
```

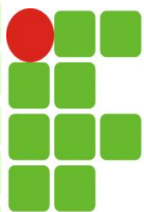


ARITMÉTICA DE PONTEIRO

- Portanto, sempre que um ponteiro for incrementado, ele apontará para a localização da memória do próximo elemento de seu tipo base.
- Sempre que ele for decrementado, apontará para a localização do elemento anterior de seu tipo base.
- Também é possível adicionar ou subtrair números inteiros a um ponteiros.

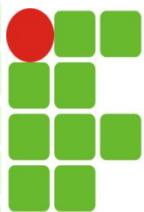
$px = px + 5;$

px passa a apontar para o quinto elemento do tipo base além daquele para o qual ele está apontando atualmente.



ARITMÉTICA DE PONTEIRO

- Aritmética de ponteiros pode ser utilizada para acessar elementos de um array, sendo na maioria das vezes mais rápido do que o acesso indexado.
- Atenção especial ao trabalhar com array do tipo char, pois quando o objeto cout recebe como argumento um ponteiro char, imprime o texto armazenado com base neste endereço.

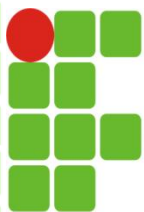


ARITMÉTICA DE PONTEIRO

■ Exemplo:

```
char texto[100];  
char *ptexto;  
ptexto = texto;  
cout << "Digite um texto qualquer: ";  
gets(texto);  
while(*ptexto)  
{  
    cout << *ptexto << " - ";  
    cout << ptexto << endl;  
    ptexto++;  
}
```

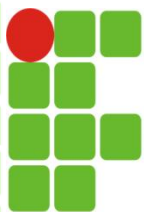
```
Digite um texto qualquer: Adilso  
A - Adilso  
d - dilso  
i - ilso  
l - lso  
s - so  
o - o
```



ARITMÉTICA DE PONTEIRO

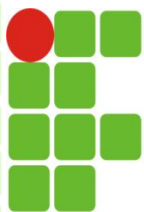
- Também é possível acessar qualquer posição dentro do array, basta para isso indicar a posição a partir do endereço atual.

```
cout << *(ptexto + 3);
```
- Este exemplo mostra a quarta posição do vetor pois o mesmo inicia em zero.
- Atenção especial no uso dos parênteses, isso se deve pois o operador `*` tem maior prioridade que o de adição, sem os parênteses ele acabaria somando 3 ao elemento atual apontado.



ARITMÉTICA DE PONTEIRO

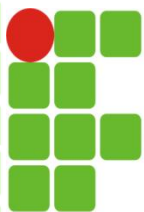
- A manipulação do tipo string também é possível com o uso de aritmética de ponteiros, para isso basta observar a forma de endereçar o ponteiro e o ponteiro será sempre do tipo char.
string texto;
char *ptexto;
ptexto = &texto[0];



ARITMÉTICA DE PONTEIRO

■ Exemplo:

```
system("chcp 1252 > nul");  
string nome = "Fulano de Tal";  
char *p = &nome[0];  
while(*p)  
{  
    cout << *p << endl;  
    p++;  
}
```



REFERÊNCIAS

- SCHILDT, Herbert. C completo e total. Rio de Janeiro: Editora Ciência Moderna LTDA 3ª edição. 2000.
- PEREIRA, Silvio do Lago. Estrutura de Dados Fundamentais: Conceitos e Aplicações, 12. Ed. São Paulo, Érica, 2008.
- LORENZI, Fabiana. MATTOS, Patrícia Noll de. CARVALHO, Tanisi Pereira de. Estrutura de Dados. São Paulo: Ed. Thomson Learning, 2007.
- VELOSO, Paulo. SANTOS, Celso dos. AZEVEDO, Paulo. FURTADO, Antonio. Estrutura de dados. Rio de Janeiro: Ed. Elsevier, 1983 27ª reimpressão.